

ApnaDairy

Complete Project Documentation

Setup, Supabase, portals, links, passwords, and file structure

Repository: github.com/mhasnat013/ApnaDairy-Version-1

Security rule: real passwords, database credentials, JWT secrets, API keys, and tokens are not stored in this public document. Use backend/.env and a secure password manager.

1 Project architecture

ApnaDairy uses React and TypeScript for the browser, FastAPI and SQLAlchemy for the API, and Supabase PostgreSQL for shared online data. The browser calls FastAPI; FastAPI validates requests, applies JWT and role guards, executes business services, and reads or writes the database.

Layer	Technology	Responsibility
Frontend	React, TypeScript, Vite, Tailwind	Website and role portals
Backend	FastAPI, SQLAlchemy, Pydantic	API, business rules, authentication
Database	Supabase PostgreSQL	Online shared data
Local demo	SQLite	Zero-configuration testing
Migrations	Alembic and supabase_schema.sql	Repeatable schema setup
Security	JWT, Passlib, role guards	Identity and authorization

IoT readings and AI freshness results are explicitly demonstration data unless real hardware and certified models are connected.

2 First-time setup in VS Code PowerShell

```
git clone https://github.com/mhasnat013/ApnaDairy-Version-1.git
Set-Location .\ApnaDairy-Version-1
git --version
node --version
npm --version
py --version
```

```
Set-Location .\frontend
npm install
Set-Location ..
```

```
py -3.12 -m venv .\backend\.venv
Set-Location .\backend
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
Set-Location ..
```

If activation is blocked: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass. Never paste .gitignore entries into PowerShell.

3 Supabase and environment setup

Supabase is online, but a new PC does not automatically know the connection. Create backend/.env from the template and fill it privately.

```
Copy-Item .\backend\.env.example .\backend\.env
code .\backend\.env
```

```
DATABASE_URL=postgresql+psycopg2://postgres:<SUPABASE_DB_PASSWORD>@<SUPABASE_HOST>:5432/postgres
JWT_SECRET=<LONG_RANDOM_SECRET>
ADMIN_EMAIL=<PRIVATE_ADMIN_EMAIL>
ADMIN_PASSWORD=<PRIVATE_ADMIN_PASSWORD>
SUPERADMIN_EMAIL=<PRIVATE_SUPERADMIN_EMAIL>
SUPERADMIN_PASSWORD=<PRIVATE_SUPERADMIN_PASSWORD>
GROQ_API_KEY=<OPTIONAL_SERVER_KEY>
DEMO_MODE=false
IOT_AI_AUTOMATION_ENABLED=false
CORS_ORIGINS=http://localhost:5174,http://127.0.0.1:5174
```

On a fresh Supabase project, run backend/supabase_schema.sql in Dashboard -> SQL Editor. For Alembic-managed history:

```
Set-Location .\backend
.\.venv\Scripts\Activate.ps1
alembic upgrade head
python -m app.seed
```

Never upload backend/.env. Rotate any exposed credentials before sharing.

4 Start project and portals

Run backend and frontend in two VS Code terminals.

Backend

```
Set-Location .\backend
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Frontend

```
Set-Location .\frontend
npm run dev -- --host 127.0.0.1 --port 5174
```

Portal	Link	Credential source
Public	http://127.0.0.1:5174/	No login
Login	http://127.0.0.1:5174/login	Seeded or registered user
Customer	http://127.0.0.1:5174/customer-dashboard	Customer account
Farmer	http://127.0.0.1:5174/farmer-dashboard	Farmer account
Business	http://127.0.0.1:5174/business-dashboard	Business account
Rider	http://127.0.0.1:5174/rider-dashboard	Approved rider account
Admin	http://127.0.0.1:5174/admin-dashboard	ADMIN_EMAIL and ADMIN_PASSWORD
Super Admin	http://127.0.0.1:5174/superadmin-dashboard	SUPERADMIN_EMAIL and SUPERADMIN_PASSWORD
API docs	http://127.0.0.1:8000/docs	Backend API

Portal passwords are not printed. They are private .env values and should be handed over through a password manager.

5 Frontend file structure

Path	Responsibility
frontend/src/main.tsx	React entry point
frontend/src/app/router.tsx	All routes
frontend/src/app/guards.tsx	Auth and role guards
frontend/src/app/portalConfig.ts	Portal navigation
frontend/src/pages/admin/	Admin dashboard and management pages
frontend/src/pages/superadmin/	Governance pages
frontend/src/pages/farmer/	Farm, batch, IoT, AI and products
frontend/src/pages/business/	Procurement and quotations
frontend/src/pages/customer/	Shop, cart, checkout, orders and reviews
frontend/src/pages/rider/	Delivery workflows
frontend/src/pages/shared/	Profile, chat and shared pages
frontend/src/pages/auth/	Login and password screens
frontend/src/components/	Reusable UI/layout/motion/3D
frontend/src/features/portal/	API functions and types
frontend/src/lib/apiClient.ts	HTTP and Bearer token client
frontend/src/stores/	Global state
frontend/src/assets/	Visual assets

6 Backend file structure

Path	Responsibility
backend/app/main.py	FastAPI app, middleware and router registration
backend/app/api/routes/auth.py	Register, login, refresh, reset, current user
backend/app/api/routes/admin.py	Admin users, farms, operations, complaints and logs
backend/app/api/routes/super_admin.py	Governance, applications and assignments
backend/app/api/routes/analytics.py	Dashboards and analytics
backend/app/api/routes/batches.py	Milk batch workflows
backend/app/api/routes/catalog.py	Farms and product catalog
backend/app/api/routes/commerce.py	Cart, orders, payments, deliveries
backend/app/api/routes/b2b.py	Business procurement
backend/app/api/routes/iot_ai.py	IoT and AI predictions
backend/app/api/routes/engagement.py	Notifications, reviews, complaints
backend/app/api/routes/support.py	Support and chat
backend/app/api/routes/uploads.py	Validated uploads
backend/app/auth/security.py	JWT and password hashing
backend/app/auth/deps.py	User, role and ownership guards
backend/app/core/config.py	Environment and CORS
backend/app/db/database.py	Engine and sessions
backend/app/models/	Database tables and enums
backend/app/schemas/	Validation models
backend/app/services/	AI, IoT and automation
backend/supabase_schema.sql	Supabase schema
backend/alembic/	Migrations
backend/tests/	Automated tests

7 Login and role protection

Login.tsx -> apiClient.ts -> POST /api/v1/auth/login
auth.py -> User lookup -> password verification
security.py -> access and refresh JWT
Authorization: Bearer <token>
deps.py -> decode token -> load User -> require_role(...)

Flow	Rule
Registration	Customer, farmer, business, rider only
Admin creation	Only app.seed with private variables
Super Admin creation	Only app.seed with private variables
Identity	GET /api/v1/auth/me
Admin API	admin or superadmin role
Super Admin API	superadmin role
Rider	Admin approval changes pending to active

Frontend links are not security boundaries. Backend role and ownership guards enforce access.

8 Verification and troubleshooting

```
Invoke-WebRequest http://127.0.0.1:8000/api/v1/health
Invoke-WebRequest http://127.0.0.1:8000/openapi.json
Set-Location .\frontend
npm run build
Set-Location ..\backend
.\.venv\Scripts\Activate.ps1
pytest
```

Check	Expected result
Schema	Tables visible in Supabase
Seed	Six roles and connected demo records
Login	Returns accessToken and user
Current user	Returns identity for Bearer token
Admin guard	Customer receives 403
Build	TypeScript and Vite pass
Git safety	.env and generated folders ignored

If packages compile, use Python 3.12/3.13. If CORS fails, update CORS_ORIGINS and restart backend. If login fails, run seed against the same DATABASE_URL used by uvicorn.

Reference: README.md, RUN-LOCALLY.md, API_CONTRACT.md, frontend/STRUCTURE.md, backend/STRUCTURE.md, backend/supabase_schema.sql, backend/.env.example.